

Contents

1	Abstract	4
2	Introduction	5
3	Context and Motivation	6
3.1	Definitions	6
3.1.1	Time series	6
3.1.2	Finance	6
3.1.3	Market	6
3.1.4	Index	6
3.1.5	Volatility	6
4	State of the Art	7
4.1	Econometric Approach: GARCH Family	7
4.2	High-Frequency Paradigms: The HAR-RV Framework	7
4.3	Machine Learning Approach	7
4.4	Deep Learning Approach	8
4.4.1	Hybrid Architectures	8
5	Theoretical Framework	9
5.1	Support Vector Regression	9
5.2	Decision Trees	9
5.3	Random Forest	9
5.4	XGBoost	9
5.5	ACF and PACF	9
5.6	Autoregressive (AR) Model	9
5.7	Moving Average (MA) Model	9
5.8	ARCH/GARCH Model	10
5.9	Recurrent Neural Network (RNN)	10
5.10	Long Short Term Memory (LSTM) Network	10
5.11	Gated Recurrent Unit (GRU) Network	10
5.12	Loss Functions	10
5.13	Stationnarity	10
5.14	Augmented-Dickey-Fuller Test	10
5.15	Khi-2 Distributions	10
5.16	Student Distributions	10
5.17	Ljung-Box Test	11
5.18	Optimization techniques	11
5.18.1	Yule-Walker Equations:	12
5.18.2	Ordinary Least Squares (OLS):	12
5.18.3	Maximum Likelihood Estimation (MLE):	13
5.18.4	Adam Optimizer	14
6	Methodology	15
6.1	Data Description	15
6.1.1	Dataset Structure	15
6.1.2	Variable Definitions	15

6.2	Data Engineering and Preprocessing	15
6.2.1	Realized Volatility Construction	15
6.2.2	Stationarity	16
6.2.2.1	Results	16
6.2.3	Splitting	16
6.2.4	Normalization	16
6.3	Hybrid Approach	17
6.3.1	Stage 1: The GARCH Filter	17
6.3.1.1	Model Specification:	17
6.3.1.2	GARCH Raw Prediction	18
6.3.2	Stage 2: Residuals analysis	18
6.3.2.1	Calculation of Residuals	18
6.3.2.2	Residuals analysis - ACF and PACF	19
6.3.2.3	Ljung-Box Test	19
6.3.3	Stage 3: GRU Residual Correction	19
6.3.3.1	Input Structure:	19
6.3.3.2	Network Architecture:	20
6.3.4	Final Prediction	20
7	Empirical Results	21
7.1	Evaluation Metrics	21
7.2	Volatility Forecasting Results	21
7.3	Interpretation	21
8	Conclusion and Future Work	21
9	References	21

Abbreviations

- ARCH: Autoregressive Conditional Heteroskedasticity
- GARCH: Generalized Autoregressive Conditional Heteroskedasticity
- LSTM: Long Short Term Memory
- GRU: Gated Recurrent Unit
- RNN: Recurrent Neural Network
- MSE: Mean Squared Error
- RMSE: Root Mean Squared Error
- MAE: Mean Absolute Error
- RV: Realized Volatility

1 Abstract

2 Introduction

3 Context and Motivation

3.1 Definitions

3.1.1 Time series

In statistics, a time series is a sequence of data points collected or recorded at successive points in time, typically at uniform intervals. Time series data is characterized by its temporal ordering, meaning that the sequence of observations is crucial for analysis. Examples of time series data include daily stock prices, monthly sales figures, annual rainfall measurements, and hourly temperature readings.

3.1.2 Finance

Finance is the study and management of money, investments, and other financial instruments. It encompasses a wide range of activities, including the allocation of assets, risk management, capital raising, and the analysis of financial markets. Finance can be broadly categorized into three main areas: personal finance, corporate finance, and public finance. It involves understanding how individuals, businesses, and governments make decisions about resource allocation over time, taking into account factors such as interest rates, inflation, and market volatility...

3.1.3 Market

In finance, a market refers to a system or environment where buyers and sellers interact to exchange goods, services, or financial instruments. Markets facilitate the trading of assets such as stocks, bonds, commodities, currencies, and derivatives. They can be physical locations, like stock exchanges (e.g., New York Stock Exchange), or virtual platforms, such as electronic trading systems.

3.1.4 Index

An index in finance is a statistical measure that represents the performance of a specific group of assets, such as stocks, bonds, or commodities. It serves as a benchmark to track the overall market or a particular segment of the market. Indices are typically calculated using a weighted average of the prices or market capitalizations of the constituent assets. Examples of well-known indices include the S&P 500, Dow Jones Industrial Average, and NASDAQ Composite.

3.1.5 Volatility

Is a statistical measure of the dispersion of returns for a given security or market index. It quantifies the degree of variation in the price of a financial instrument over time, reflecting the uncertainty or risk associated with its value. High volatility indicates significant price fluctuations, while low volatility suggests more stable prices. Volatility is commonly measured using metrics such as standard deviation or variance of returns, and it plays a crucial role in risk management, option pricing, and investment strategies.

4 State of the Art

The problem of volatility forecasting has evolved from parametric econometric models, grounded in statistical theory, to non-parametric deep learning architectures that prioritize predictive accuracy. This section outlines the industry standards.

4.1 Econometric Approach: GARCH Family

The Generalized Autoregressive Conditional Heteroskedasticity (GARCH) family, introduced by Bollerslev (1986) as an extension of Engle’s (1982) ARCH model, has served as the standard in quantitative finance for decades. These models are designed to capture the *volatility clustering*—the phenomenon where large changes in asset prices tend to be followed by large changes, and small changes by small changes (Mandelbrot, 1963).

The standard GARCH(1,1) specifies the conditional variance σ_t^2 as a linear function of past squared residuals (innovations) and past variance:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2 \quad (1)$$

4.2 High-Frequency Paradigms: The HAR-RV Framework

With the availability of high-frequency tick data, the focus shifted from estimating latent volatility (via daily returns in GARCH) to modeling *Realized Volatility (RV)* directly as an observable process, providing a noise-reduced proxy for the true integrated variance.

The Heterogeneous Autoregressive (HAR-RV) model, proposed by Corsi (2009), is the specific industry baseline for Realized Volatility. Unlike GARCH, which treats volatility as hidden, HAR-RV models it as a long-memory process resulting from the interaction of heterogeneous market participants with different time horizons (high-frequency traders, portfolio managers, and institutional investors).

The model posits that tomorrow’s volatility is a linear function of volatilities realized over daily (d), weekly (w), and monthly (m) horizons:

$$RV_{t+1} = \beta_0 + \beta_D RV_t^{(d)} + \beta_W RV_t^{(w)} + \beta_M RV_t^{(m)} + \epsilon_{t+1} \quad (2)$$

where the multi-period components are averages of past daily realized volatilities:

$$RV_t^{(w)} = \frac{1}{5} \sum_{j=1}^5 RV_{t-j+1}, \quad RV_t^{(m)} = \frac{1}{22} \sum_{j=1}^{22} RV_{t-j+1} \quad (3)$$

The HAR-RV model is favored in the industry for its simplicity and its ability to mimic “long memory” persistence without the mathematical complexity of fractional integration (FIGARCH).

4.3 Machine Learning Approach

To overcome the parametric constraints of GARCH and the linearity of HAR-RV, researchers initially turned to classical Machine Learning (ML) algorithms such as Support Vector Regression (SVR) and Random Forests. These non-parametric models introduced the capability to map non-linear relationships between covariates and volatility targets without making strong distributional assumptions.

However, standard ML models (like Random Forests) treat time-series data as static observations, ignoring the temporal ordering and sequential dependencies inherent in financial data. This limitation necessitates complex "feature engineering" (e.g., manually creating lag features) and often fails to capture the long-term temporal context required for accurate forecasting.

4.4 Deep Learning Approach

To address the sequence modeling limitations of classical ML, the state-of-the-art has shifted toward Deep Learning, specifically Recurrent Neural Networks (RNNs). RNNs are designed to process sequential data by maintaining an internal "hidden state" that acts as a memory of previous inputs.

Relevance to this study: The GRU is particularly pertinent to financial time series forecasting. Empirical studies suggest that GRUs often match or exceed the performance of LSTMs on volatility tasks while being computationally more efficient and less prone to overfitting on smaller datasets typical in finance.

4.4.1 Hybrid Architectures

The current frontier in academic literature is the **Hybrid Approach**. These architectures combine the strengths of econometrics (GARCH) and deep learning (GRU/LSTM). A common methodology involves using a GARCH model to filter the linear "trend" of volatility and feeding the residuals (the non-linear noise) into a Neural Network. This allows the Deep Learning model to focus solely on the complex, non-linear patterns that the statistical model failed to capture, theoretically offering the best of both worlds.

5 Theoretical Framework

5.1 Support Vector Regression

5.2 Decision Trees

5.3 Random Forest

5.4 XGBoost

5.5 ACF and PACF

5.6 Autoregressive (AR) Model

Is a type of statistical model used for analyzing and forecasting time series data. The AR model assumes that the current value of a time series is linearly dependent on its previous values (lags) and a stochastic error term. The model is defined by the order p , which indicates the number of lagged observations included in the model. The mathematical representation of an AR(p) model is given by:

$$X_t = c + \sum_{i=1}^p \phi_i X_{t-i} + \epsilon_t \quad (4)$$

where:

- X_t is a Random variable representing the value of the time series at time t .
- c is a constant term, assumed 0 for centered Data.
- ϕ_i are the coefficients of the model, representing the influence of past values on the current value.
- X_{t-i} are the lagged values of the time series.
- $\{\epsilon_t\} \sim WN(0, \sigma^2)$ is an error term, assumed to be normally distributed with mean zero and constant variance and zero covariance.

5.7 Moving Average (MA) Model

Is a statistical model used for analyzing and forecasting time series data. The MA model assumes that the current value of a time series is linearly dependent on past error terms (shocks) and a stochastic error term. The model is defined by the order q , which indicates the number of lagged error terms included in the model. The mathematical representation of an MA(q) model is given by:

$$X_t = \mu + \sum_{i=1}^q \theta_i \epsilon_{t-i} + \epsilon_t \quad (5)$$

where:

- X_t is the value of the time series at time t .
- μ is the mean of the time series.

- θ_i are the coefficients of the model, representing the influence of past error terms on the current value.
- ϵ_{t-i} are the lagged error terms (shocks) of the time series.
- ϵ_t is a white noise error term, assumed to be normally distributed with mean zero and constant variance.

5.8 ARCH/GARCH Model

5.9 Recurrent Neural Network (RNN)

5.10 Long Short Term Memory (LSTM) Network

5.11 Gated Recurrent Unit (GRU) Network

5.12 Loss Functions

5.13 Stationnarity

It represents a property of a time series whereby its statistical characteristics, such as mean, variance, and autocovariance, remain constant over time. In other words, a stationary time series does not exhibit trends, seasonal patterns, or other forms of non-stationarity that could affect its behavior over time.

5.14 Augmented-Dickey-Fuller Test

5.15 Khi-2 Distributions

The Chi-squared (χ^2) distribution is a continuous probability distribution that arises in statistics when a variable is the sum of the squares of k independent standard normal random variables. It is widely used in hypothesis testing, particularly in tests of independence and goodness-of-fit. The probability density function (PDF) of the Chi-squared distribution with k degrees of freedom is given by:

$$f(x; k) = \frac{1}{2^{k/2}\Gamma(k/2)} x^{(k/2)-1} e^{-x/2} \quad \text{for } x > 0 \quad (6)$$

where:

- k is the degrees of freedom.
- Γ is the gamma function, which generalizes the factorial function.
- e is the base of the natural logarithm.

5.16 Student Distributions

The Student's t-distribution, commonly referred to as the t-distribution, is a probability distribution that arises when estimating the mean of a normally distributed population in situations where the sample size is small and the population standard deviation is unknown. It was introduced by William Sealy Gosset under the pseudonym "Student"

in 1908. The probability density function (PDF) of the Student's t-distribution with ν degrees of freedom is given by:

$$f(t; \nu) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi} \Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{t^2}{\nu}\right)^{-\frac{\nu+1}{2}} \quad \text{for } -\infty < t < \infty \quad (7)$$

where:

- ν is the degrees of freedom.
- Γ is the gamma function.
- π is the mathematical constant Pi (approximately 3.14159).
- e is the base of the natural logarithm.

5.17 Ljung-Box Test

The Ljung-Box test is a statistical test used to determine whether there is significant autocorrelation in a time series dataset. It tests the null hypothesis H_0 that the data are independently distributed against the alternative hypothesis H_1 that there is autocorrelation at one or more lags. The test statistic is calculated as follows:

$$Q = n(n+2) \sum_{k=1}^h \frac{\hat{\rho}_k^2}{n-k} \quad (8)$$

where:

- Q is the Ljung-Box test statistic.
- n is the number of observations in the time series.
- h is the number of lags being tested.
- $\hat{\rho}_k$ is the sample autocorrelation at lag k .

Once we compute the Q , we suppose that $Q \sim \chi^2(h)$ under the null hypothesis. We can then compare the computed Q to the critical value from the chi-squared distribution with h degrees of freedom. We compute the p-value associated with the test statistic Q . If the p-value is less than a chosen significance level (0.05), we reject the null hypothesis, indicating that there is significant autocorrelation in the time series. We have:

$$p\text{-value} = P(\chi^2(h) \geq Q) = \int_Q^\infty \frac{1}{2^{h/2}\Gamma(h/2)} x^{(h/2)-1} e^{-x/2} dx \quad (9)$$

5.18 Optimization techniques

This section describes the different optimization techniques used to estimate the parameters of the different models I have used throughout this project.

5.18.1 Yule-Walker Equations:

We use Yule-Walker equations to estimate the parameters of the AR model. The Yule-Walker equations relate the autocovariances of a stationary time series to the parameters of an AR model. For an AR(p) model, the Yule-Walker equations can be expressed as:

The Yule-Walker equations are given by:

$$X_t = \sum_{i=1}^p \phi_i X_{t-i} + \epsilon_t \quad (10)$$

$$X_t X_{t-k} = \sum_{i=1}^p \phi_i X_{t-i} X_{t-k} + \epsilon_t X_{t-k} \quad (11)$$

$$E[X_t X_{t-k}] = \sum_{i=1}^p E[\phi_i X_{t-i} X_{t-k}] + E[\epsilon_t X_{t-k}], \quad E[\epsilon_t X_{t-k}] = 0 \quad (12)$$

$$\gamma(k) = \sum_{i=1}^p \phi_i \gamma(k-i) \quad (13)$$

where:

- $\gamma(k)$ is the autocovariance function at lag k .
- ϕ_i are the AR model parameters to be estimated.
- σ^2 is the variance of the white noise error term.

Matrix form:

$$\begin{bmatrix} \gamma(1) \\ \gamma(2) \\ \vdots \\ \gamma(p) \end{bmatrix} = \begin{bmatrix} \gamma(0) & \gamma(1) & \cdots & \gamma(p-1) \\ \gamma(1) & \gamma(0) & \cdots & \gamma(p-2) \\ \vdots & \vdots & \ddots & \vdots \\ \gamma(p-1) & \gamma(p-2) & \cdots & \gamma(0) \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \\ \vdots \\ \phi_p \end{bmatrix} \quad (14)$$

5.18.2 Ordinary Least Squares (OLS):

We can also estimate the parameters of the AR model using the Ordinary Least Squares (OLS) method, considering the AR model as a linear regression of the current value on its past values.

We have the AR(p) model in matrix form as:

$$Y = X\phi + \epsilon \quad (15)$$

where:

$$\begin{bmatrix} X_{p+1} \\ X_{p+2} \\ \vdots \\ X_T \end{bmatrix} = \begin{bmatrix} X_p & X_{p-1} & \cdots & X_1 \\ X_{p+1} & X_p & \cdots & X_2 \\ \vdots & \vdots & \ddots & \vdots \\ X_{T-1} & X_{T-2} & \cdots & X_{T-p} \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \\ \vdots \\ \phi_p \end{bmatrix} + \begin{bmatrix} \epsilon_{p+1} \\ \epsilon_{p+2} \\ \vdots \\ \epsilon_T \end{bmatrix} \quad (16)$$

We then minimize the sum of squared residuals to estimate the parameters:

$$S(\phi) = \sum_{t=p+1}^T (x_t - \sum_{i=1}^p \hat{\phi}_i x_{t-i})^2 = (Y - X\hat{\phi})^T(Y - X\hat{\phi}) \quad (17)$$

We derivate with respect to ϕ and set it to zero to find the optimal parameters:

$$\frac{\partial S(\phi)}{\partial \phi} = -2X^T(Y - X\hat{\phi}) = 0 \quad (18)$$

$$X^T X \hat{\phi} = X^T Y \quad (19)$$

We multiply both sides by the inverse of $X^T X$ to isolate $\hat{\phi}$:

$$\hat{\phi} = (X^T X)^{-1} X^T Y \quad (20)$$

We just need to compute this equation to get the OLS estimates of the AR model parameters.

5.18.3 Maximum Likelihood Estimation (MLE):

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate the parameters of a model by maximizing the likelihood function, which measures how well the model explains the observed data. It is defined as:

$$L(\theta; x) = P(x|\theta) = \prod_{i=1}^n P(x_i|\theta) \quad (21)$$

where:

- $L(\theta; x)$ is the likelihood function.
- θ represents the parameters of the model to be estimated.
- x represents the observed data.
- $P(x|\theta)$ is the probability of observing the data x given the parameters θ .

To find the MLE estimates of the parameters, we typically work with the log-likelihood function, which is the natural logarithm of the likelihood function:

$$\ell(\theta; x) = \ln L(\theta; x) \quad (22)$$

We then maximize the log-likelihood function with respect to the parameters θ :

$$\hat{\theta} = \arg \max_{\theta} \ell(\theta; x) \quad (23)$$

This is often done by taking the derivative of the log-likelihood function with respect to the parameters, setting it to zero, and solving for the parameters.

5.18.4 Adam Optimizer

Adaptive Moment Estimation (Adam) is an optimization algorithm designed for training deep learning models. It combines the advantages of two other popular optimization techniques: Adaptive Gradient Algorithm (AdaGrad) and Root Mean Square Propagation (RMSProp). Adam computes adaptive learning rates for each parameter by maintaining running averages of both the gradients and their squared values. The Adam update rules are as follows:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (24)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (25)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (26)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (27)$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (28)$$

where:

- g_t is the gradient of the loss function with respect to the parameters θ at time step t .
- m_t and v_t are the first and second moment estimates (mean and uncentered variance) of the gradients.
- β_1 and β_2 are hyperparameters that control the decay rates of these moment estimates, typically set to 0.9 and 0.999, respectively.
- η is the learning rate.
- ϵ is a small constant added for numerical stability, often set to 10^{-8} .
- $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates of the first and second moments.

6 Methodology

6.1 Data Description

The dataset utilized in this project is a high-frequency financial time series spanning a period of approximately 13 years. The data was collected between **January 22, 2008**, and **May 6, 2021**. The total sample size consists of $N = 2,070,834$ observations, indexed by a datetime stamp representing the specific trading interval.

6.1.1 Dataset Structure

The data is structured as a time-indexed DataFrame, with a memory footprint of approximately 126.4 MB. with intraday resolution (1 minute), covering trading sessions typically ranging from 07:30 to 14:00. Each observation vector x_t at time t is defined by seven distinct features.

6.1.2 Variable Definitions

Table 1 provides a summary of the variables included in the dataset, their data types, and their descriptions.

Table 1: Description of Variables in the Dataset			
Variable	Data Type	Notation	Description
open	Float64	O_t	The price of the asset at the beginning of the time interval t .
high	Float64	H_t	The highest price reached during the interval t .
low	Float64	L_t	The lowest price reached during the interval t .
close	Float64	C_t	The price of the asset at the end of the time interval t .
volume	Int64	V_t	The total number of units traded during the interval.
barCount	Int64	B_t	The number of distinct trades (ticks) that occurred during the interval.
average	Float64	A_t	The average price of the asset during the interval.

6.2 Data Engineering and Preprocessing

To transform the raw high-frequency data into a target variable suitable for daily volatility forecasting, we constructed the **Realized Volatility (RV)** metric using a "sum-of-squares" approach that accounts for both intraday trading and overnight gaps.

6.2.1 Realized Volatility Construction

The volatility target was constructed in three steps:

1. **Intraday Variance (RV_{intra}):** The 1-minute data was resampled to 5-minute intervals to mitigate microstructure noise (bid-ask bounce¹). If $r_{t,i}$ represents the log-return at the i -th 5-minute interval on day t , the intraday variance is:

$$RV_{t,intra} = \sum_{i=1}^M r_{t,i}^2 \quad (29)$$

2. **Overnight Variance (RV_{night}):** Asset prices often jump between the market close of day $t - 1$ and the open of day t . We capture this using the squared overnight return:

$$RV_{t,night} = \ln \left(\frac{O_t}{C_{t-1}} \right)^2 \quad (30)$$

3. **Total Realized Volatility:** The total daily realized variance is the sum of these components. The final target variable (volatility) is the square root of the scaled variance:

$$\sigma_{t,realized} = \sqrt{10000 \times (RV_{t,intra} + RV_{t,night})} \quad (31)$$

6.2.2 Stationarity

We performed an Augmented Dickey-Fuller (ADF) test on the daily log returns to ensure stationarity, rejecting the null hypothesis of a unit root ($p < 0.05$).

6.2.2.1 Results

ADF Statistic: -14.315982130207596

p-value: 1.1627203853193956e-26

Critical Values:

1%: -3.4323164516352445

5%: -2.8624088629359186

10%: -2.5672325132363225

6.2.3 Splitting

The dataset was split into training (90%) and testing (10%) sets using a temporal split non-shuffled to preserve the time-series structure.

6.2.4 Normalization

To facilitate gradient descent convergence, input features for the Neural Network were standardized using Z-score normalization:

$$z = \frac{x - \mu}{\sigma} \quad (32)$$

where μ and σ are the mean and standard deviation of the training set, respectively.

¹Fluctuations caused by traders alternating between the bid and ask price.

6.3 Hybrid Approach

The proposed solution implements a **Residual Learning Framework**. The core hypothesis is that volatility σ_t comprises a linear component (explainable by econometric theory) and a non-linear residual component (explainable by deep learning).

6.3.1 Stage 1: The GARCH Filter

We first train a GARCH(1,1) model to capture the linear autocorrelation of the returns. The GARCH model outputs a baseline forecast $\hat{\sigma}_{GARCH,t}$.

6.3.1.1 Model Specification:

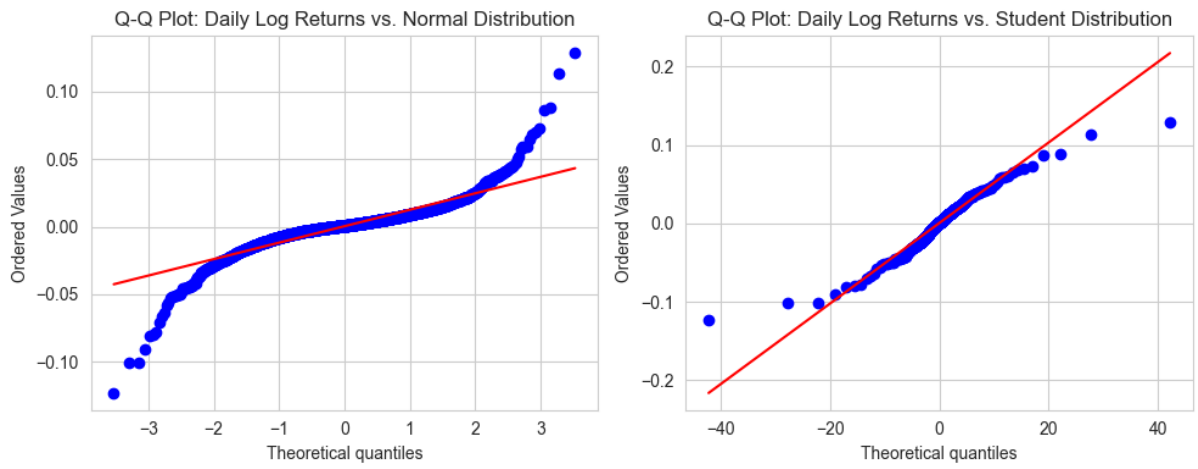


Figure 1: Q-Q Plot of GARCH Residuals vs Theoretical Distributions

Justification:

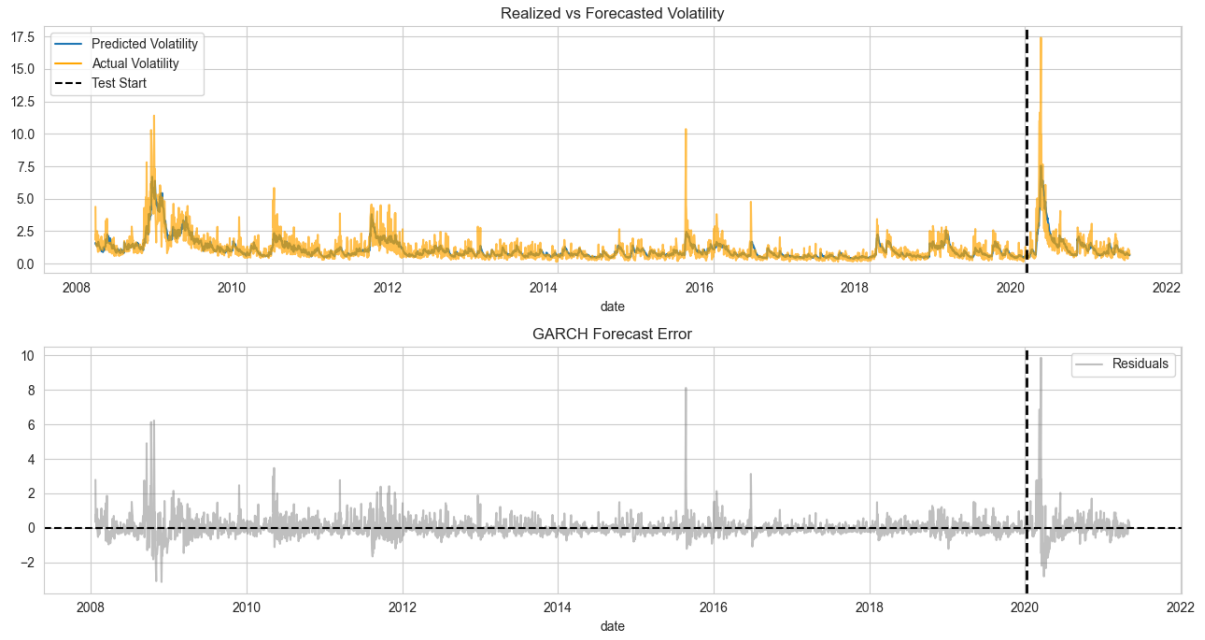


Figure 2: GARCH(1,1) Volatility Forecast and Realized Volatility

6.3.1.2 GARCH Raw Prediction

6.3.2 Stage 2: Residuals analysis

6.3.2.1 Calculation of Residuals The residuals ϵ_t were computed as the difference between the realized volatility and the GARCH forecast:

$$\epsilon_t = \sigma_{t,realized} - \hat{\sigma}_{GARCH,t} \quad (33)$$

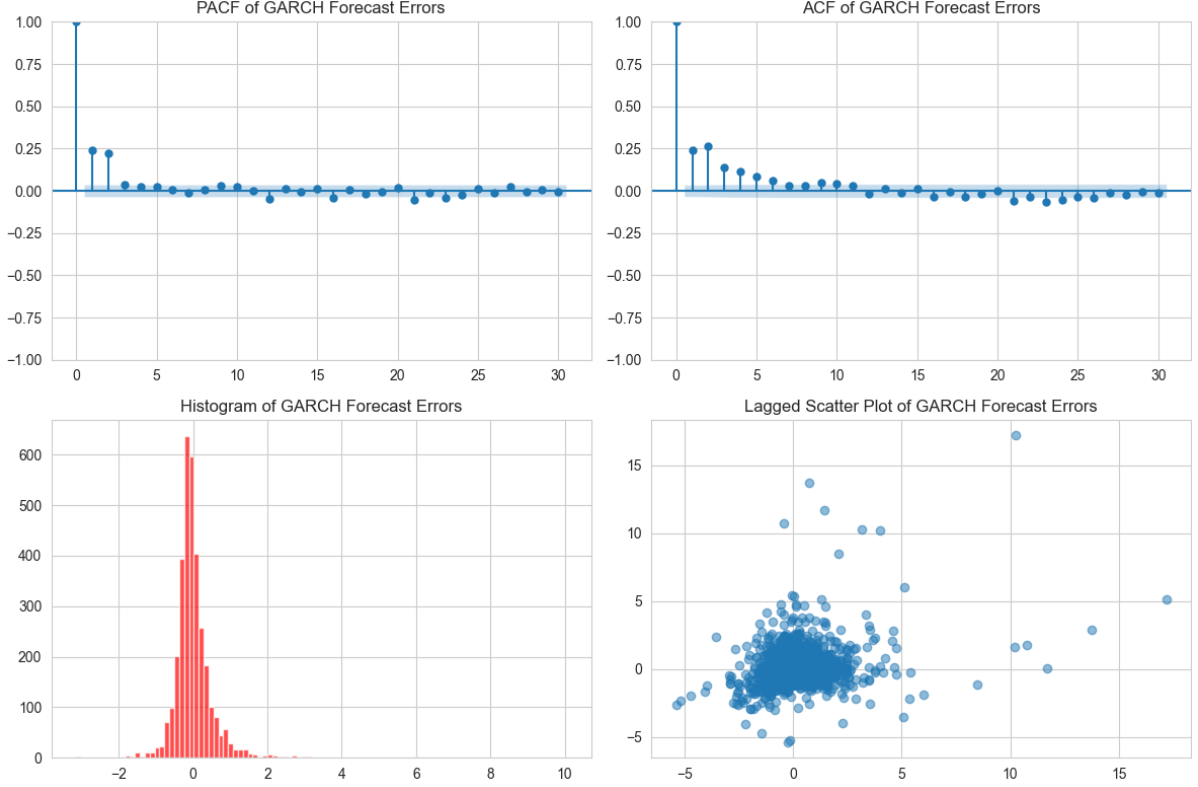


Figure 3: Analysis of GARCH Residuals

6.3.2.2 Residuals analysis - ACF and PACF We observe significant autocorrelation in the residuals up to lag 2 (PACF plot) and up to lag 5 (ACF plot), indicating that GARCH has not fully captured the temporal dependencies in volatility. Justifying the use of a sequential model like GRU to learn these patterns.

6.3.2.3 Ljung-Box Test

lag	lb_stat	lb_pvalue
5	562.966721	2.023881e-119
10	594.640592	2.477339e-121
50	751.438069	7.242546e-126
100	818.981855	2.700238e-113

The Ljung-Box test confirms the presence of autocorrelation in the residuals ($p < 0.05$), validating our approach.

6.3.3 Stage 3: GRU Residual Correction

The role of the GRU network is to model the non-linear structure in the GARCH residuals ϵ_t .

6.3.3.1 Input Structure: The model utilizes a lookback window of $k = 20$ days . The input vector X_t at each timestep includes three features:

1. The past GARCH residuals $(\epsilon_{t-1} \dots \epsilon_{t-k})$.

2. The past daily log-returns $(r_{t-1} \dots r_{t-k})$.
3. The target volatility $(\sigma_{t-1} \dots \sigma_{t-k})$.

6.3.3.2 Network Architecture: The GRU architecture was optimized via experimentation (as detailed in the code implementation). The final configuration is defined as follows:

Table 2: Hybrid GRU Network Hyperparameters

Layer/Parameter	Specification
Input Shape	(Lookback=20, Features=3)
GRU Layer	64 Units, Activation: tanh , Return Sequences: False
Dropout	Rate: 0.3 (to prevent overfitting)
Dense Layer 1	32 Units, Activation: relu
Output Layer	1 Unit, Activation: relu
Optimizer	Adam
Loss Function	Mean Squared Error (MSE)

6.3.4 Final Prediction

The final hybrid forecast is the summation of the statistical baseline and the neural network’s correction:

$$\hat{\sigma}_{Hybrid,t} = \hat{\sigma}_{GARCH,t} + \hat{\epsilon}_{GRU,t} \quad (34)$$

7 Empirical Results

7.1 Evaluation Metrics

7.2 Volatility Forecasting Results

7.3 Interpretation

8 Conclusion and Future Work

9 References